

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <math.h>

#include <limits.h>


#define NUM_W 4

#define NUM_Z 6

#define MAX_NODES (NUM_W+NUM_Z)

#define NUM_RESOURCES 4

#define MAX_TICKS 20

#define VEHICLES_PER_TICK 3

#define INF 1e9

#define W_URGENCY .45

#define W_DEMAND .25

#define W_DISTANCE .20

#define W_AGING .10

#define URGENCY_MAX 10.0

#define DEMAND_SCALE 300.0

#define AGING_SCALE ((double)MAX_TICKS)


static const char
*RESOURCE_NAMES[NUM_RESOURCES]={"Food","Medicine","Water","ShelterKits"};

typedef enum
{STRAT_URGENCY,STRAT_DEMAND,STRAT_DISTANCE,STRAT_HYBRID,STRAT_
COUNT} Strategy;

static const char *STRATEGY_NAMES[STRAT_COUNT]={"Urgency-First","Demand-
First","Distance-First","Hybrid-Weighted"};


typedef struct{int id;char name[32];int supply[NUM_RESOURCES];} Warehouse;

```

```

typedef struct{int id;char name[32];int demand[NUM_RESOURCES];int
initial_total_demand;int urgency;int waiting_ticks;int first_served_tick;int total_delivered;}
Zone;

typedef struct{int tick,zone_id,delta;const char *reason;} UrgencyEvent;

typedef struct{int zone_id;double score;} HeapNode;

typedef struct{HeapNode data[NUM_Z];int size;} MaxHeap;

typedef struct{double
avg_response_time,fairness_index,demand_satisfaction,supply_utilization;int
starved_zones,ticks_to_finish;} Metrics;

static UrgencyEvent EVENTS[]={
    {2,1,4,"Aftershock strikes Hillcrest Village"},
    {3,3,5,"Floodwaters rising at Lowland Farms"},
    {5,4,6,"Fire spreading toward Forest Settlement"},
    {7,0,3,"Secondary aftershock hits Riverside Town"}
};

static const int NUM_EVENTS=sizeof(EVENTS)/sizeof(EVENTS[0]);

static Warehouse warehouses[NUM_W];

static Zone zones[NUM_Z];

static double
base_dist[MAX_NODES][MAX_NODES],dist_from_w[NUM_W][MAX_NODES];

static int
base_cap[MAX_NODES][MAX_NODES],has_edge[MAX_NODES][MAX_NODES];

static int
cap_remaining[MAX_NODES][MAX_NODES],parent_from_w[NUM_W][MAX_NODES];

static inline int zone_node(int z){return NUM_W+z;}

static void add_edge(int u,int v,double d,int c){
    has_edge[u][v]=has_edge[v][u]=1;
    base_dist[u][v]=base_dist[v][u]=d;
    base_cap[u][v]=base_cap[v][u]=c;
}

```

```

static void build_network(void){
    memset(has_edge,0,sizeof(has_edge));
    for(int i=0;i<MAX_NODES;i++)
        for(int j=0;j<MAX_NODES;j++)
            base_dist[i][j]=(i==j)?0.0:INF;

    add_edge(0,zone_node(0),12,18);
    add_edge(0,zone_node(1),20,10);
    add_edge(0,zone_node(2),8,20);
    add_edge(1,zone_node(1),10,14);
    add_edge(1,zone_node(3),15,12);
    add_edge(1,zone_node(4),25,8);
    add_edge(2,zone_node(3),9,15);
    add_edge(2,zone_node(4),18,9);
    add_edge(2,zone_node(5),14,11);
    add_edge(3,zone_node(5),6,20);
    add_edge(3,zone_node(2),22,8);
    add_edge(0,1,30,25);
    add_edge(zone_node(0),zone_node(1),7,6);
}

```

```

static void dijkstra(int src,double dist[],int parent[]){
    int visited[MAX_NODES]={0};
    for(int i=0;i<MAX_NODES;i++){dist[i]=INF;parent[i]=-1;}
    dist[src]=0;

    for(int k=0;k<MAX_NODES;k++){
        int u=-1;double best=INF;
        for(int i=0;i<MAX_NODES;i++)

```

```

        if(!visited[i]&&dist[i]<best){best=dist[i];u=i;}
    if(u==-1)break;
    visited[u]=1;
    for(int v=0;v<MAX_NODES;v++)
        if(has_edge[u][v]&&!visited[v]){
            double nd=dist[u]+base_dist[u][v];
            if(nd<dist[v]){dist[v]=nd;parent[v]=u;}
        }
    }
}

```

```

static void precompute_all_paths(void){
    for(int w=0;w<NUM_W;w++)
        dijkstra(w,dist_from_w[w],parent_from_w[w]);
}

```

```

static int path_bottleneck(int w,int dest){
    if(dest!=w&&parent_from_w[w][dest]==-1)return 0;
    int b=INT_MAX,cur=dest;
    while(cur!=w){
        int p=parent_from_w[w][cur];
        if(p==-1)return 0;
        if(cap_remaining[p][cur]<b)b=cap_remaining[p][cur];
        cur=p;
    }
    return b;
}

```

```

static void consume_path_capacity(int w,int dest,int qty){
    int cur=dest;

```

```

while(cur!=w){
    int p=parent_from_w[w][cur];
    cap_remaining[p][cur]-=qty;
    cap_remaining[cur][p]-=qty;
    cur=p;
}
}

```

```

static void heap_init(MaxHeap *h){h->size=0;}

```

```

static void heap_swap(HeapNode *a,HeapNode *b){
    HeapNode t=*a;*a=*b;*b=t;
}

```

```

static void heap_sift_up(MaxHeap *h,int i){
    while(i>0){
        int p=(i-1)/2;
        if(h->data[p].score>=h->data[i].score)break;
        heap_swap(&h->data[p],&h->data[i]);
        i=p;
    }
}

```

```

static void heap_sift_down(MaxHeap *h,int i){
    while(1){
        int l=2*i+1,r=2*i+2,m=i;
        if(l<h->size&&h->data[l].score>h->data[m].score)m=l;
        if(r<h->size&&h->data[r].score>h->data[m].score)m=r;
        if(m==i)break;
        heap_swap(&h->data[i],&h->data[m]);
    }
}

```

```

        i=m;
    }
}

static void heap_push(MaxHeap *h,HeapNode n){
    h->data[h->size]=n;
    heap_sift_up(h,h->size++);
}

static HeapNode heap_pop(MaxHeap *h){
    HeapNode n=h->data[0];
    h->data[0]=h->data[--h->size];
    heap_sift_down(h,0);
    return n;
}

static void init_scenario(void){
    static const char *wnames[NUM_W]={"Central Hub","North Depot","South Depot","Port Warehouse"};

    static const int wsupply[NUM_W][NUM_RESOURCES]={
        {120,40,90,30},{90,30,70,20},{100,35,80,25},{120,45,90,35}
    };

    static const char *znames[NUM_Z]={
        "Riverside Town","Hillcrest Village","Downtown District",
        "Lowland Farms","Forest Settlement","Coastal Camp"
    };

    static const int zdemand[NUM_Z][NUM_RESOURCES]={
        {80,30,60,20},{40,20,30,15},{120,50,90,35},
        {60,25,45,18},{30,15,20,10},{100,45,80,30}
    };

    static const int zurgency[NUM_Z]={6,4,7,5,3,8};

```

```

for(int i=0;i<NUM_W;i++){
    warehouses[i].id=i;
    strncpy(warehouses[i].name,wnames[i],31);
    for(int r=0;r<NUM_RESOURCES;r++)
        warehouses[i].supply[r]=wsupply[i][r];
}

```

```

for(int i=0;i<NUM_Z;i++){
    zones[i].id=i;
    strncpy(zones[i].name,znames[i],31);
    int total=0;
    for(int r=0;r<NUM_RESOURCES;r++){
        zones[i].demand[r]=zdemand[i][r];
        total+=zdemand[i][r];
    }
    zones[i].initial_total_demand=total;
    zones[i].urgency=zurgency[i];
    zones[i].waiting_ticks=0;
    zones[i].first_served_tick=-1;
    zones[i].total_delivered=0;
}
reset_tick_capacity();
}

```

```

static void reset_tick_capacity(void){
    for(int i=0;i<MAX_NODES;i++)
        for(int j=0;j<MAX_NODES;j++)
            cap_remaining[i][j]=base_cap[i][j];
}

```

```

static int zone_remaining_demand(int z){
    int total=0;
    for(int r=0;r<NUM_RESOURCES;r++)total+=zones[z].demand[r];
    return total;
}

static int warehouse_total_supply(int w){
    int total=0;
    for(int r=0;r<NUM_RESOURCES;r++)total+=warehouses[w].supply[r];
    return total;
}

static double nearest_supplying_distance(int z){
    double best=INF;
    int zn=zone_node(z);
    for(int w=0;w<NUM_W;w++)
        if(warehouse_total_supply(w)>0&&dist_from_w[w][zn]<best)
            best=dist_from_w[w][zn];
    return best;
}

static double compute_score(int z,Strategy s){
    double aging=zones[z].waiting_ticks*1.5;
    double nd=nearest_supplying_distance(z);
    if(nd>=INF)nd=999;

    switch(s){
        case STRAT_URGENCY:return zones[z].urgency*10.0+aging;
        case STRAT_DEMAND:return zone_remaining_demand(z)+aging;
    }
}

```



```

case STRAT_DISTANCE: return 1000.0/(nd+1.0)+aging;
case STRAT_HYBRID: {
    double u=zones[z].urgency/URGENCY_MAX*100;
    double d=zone_remaining_demand(z)/DEMAND_SCALE*100;
    double dis=100/(nd/10.0+1);
    double a=zones[z].waiting_ticks/AGING_SCALE*100;
    if(d>100)d=100;
    return W_URGENCY*u+W_DEMAND*d+W_DISTANCE*dis+W_AGING*a;
}
default: return 0;
}
}

```

```

static void dispatch_to_zone(int z){
    int zn=zone_node(z),order[NUM_W];

    for(int i=0;i<NUM_W;i++)order[i]=i;
    for(int i=0;i<NUM_W-1;i++)
        for(int j=i+1;j<NUM_W;j++)
            if(dist_from_w[order[j]][zn]<dist_from_w[order[i]][zn]){
                int t=order[i];order[i]=order[j];order[j]=t;
            }

    for(int oi=0;oi<NUM_W&&zone_remaining_demand(z)>0;oi++){
        int w=order[oi];
        if(warehouse_total_supply(w)<=0)continue;

        int b=path_bottleneck(w,zn);
        if(b<=0)continue;
    }
}

```

```

int ship=b,total=0;
for(int r=0;r<NUM_RESOURCES&&ship>0;r++){
    if(zones[z].demand[r]<=0||warehouses[w].supply[r]<=0)continue;
    int q=zones[z].demand[r];
    if(warehouses[w].supply[r]<q)q=warehouses[w].supply[r];
    if(ship<q)q=ship;
    if(q<=0)continue;

    zones[z].demand[r]-=q;
    warehouses[w].supply[r]-=q;
    ship-=q;
    total+=q;
    zones[z].total_delivered+=q;
}
if(total>0){
    consume_path_capacity(w,zn,total);
    zones[z].waiting_ticks=0;
}
}
}

```

```

static Metrics run_strategy(Strategy s){
    init_scenario();
    int finished=MAX_TICKS,done=0;

    for(int tick=0;tick<MAX_TICKS&&!done;tick++){
        for(int e=0;e<NUM_EVENTS;e++){
            if(EVENTS[e].tick==tick){
                int z=EVENTS[e].zone_id;
                zones[z].urgency+=EVENTS[e].delta;
            }
        }
    }
}

```

```

        if(zones[z].urgency>10)zones[z].urgency=10;
    }

    reset_tick_capacity();
    MaxHeap pq;
    heap_init(&pq);

    for(int z=0;z<NUM_Z;z++)
        if(zone_remaining_demand(z)>0)
            heap_push(&pq,(HeapNode){z,compute_score(z,s)});

    int used=0;
    while(pq.size>0&&used<VEHICLES_PER_TICK){
        HeapNode n=heap_pop(&pq);
        int before=zones[n.zone_id].total_delivered;
        dispatch_to_zone(n.zone_id);
        used++;
        if(zones[n.zone_id].first_served_tick==-1&&
            zones[n.zone_id].total_delivered>before)
            zones[n.zone_id].first_served_tick=tick;
    }

    done=1;
    for(int z=0;z<NUM_Z;z++)
        if(zone_remaining_demand(z)>0){
            zones[z].waiting_ticks++;
            done=0;
        }

    if(done)finished=tick+1;

```

```
}
```

```
Metrics m={0};
```

```
m.ticks_to_finish=finished;
```

```
double rs=0,sx=0,sx2=0;
```

```
int delivered=0,demand=0,supply=280+210+240+290;
```

```
for(int z=0;z<NUM_Z;z++){
```

```
    delivered+=zones[z].total_delivered;
```

```
    demand+=zones[z].initial_total_demand;
```

```
    if(zones[z].total_delivered==0)m.starved_zones++;
```

```
    if(zones[z].first_served_tick>=0)
```

```
        rs+=zones[z].first_served_tick;
```

```
    else rs+=MAX_TICKS;
```

```
    double x=(double)zones[z].total_delivered/zones[z].initial_total_demand;
```

```
    if(x>1)x=1;
```

```
    sx+=x;
```

```
    sx2+=x*x;
```

```
}
```

```
m.avg_response_time=rs/NUM_Z;
```

```
m.fairness_index=sx2>0?(sx*sx)/(NUM_Z*sx2):1;
```

```
m.demand_satisfaction=100.0*delivered/demand;
```

```
m.supply_utilization=100.0*delivered/supply;
```

```
return m;
```

```
}
```

```

static void print_scenario_summary(void){

printf("=====
=====\\n");

printf(" DISASTER-RELIEF NETWORK\\n");

printf("=====
=====\\n");

printf(" Warehouses:\\n");

init_scenario();

for(int w=0;w<NUM_W;w++){
printf(" [W%d] %-16s Supply: ",w,warehouses[w].name);
for(int r=0;r<NUM_RESOURCES;r++)
printf("%s=%d ",RESOURCE_NAMES[r],warehouses[w].supply[r]);
printf("\\n");
}

printf("\\n Disaster Zones:\\n");
for(int z=0;z<NUM_Z;z++){
printf(" [Z%d] %-18s Urgency=%-2d Demand: ",
z,zones[z].name,zones[z].urgency);
for(int r=0;r<NUM_RESOURCES;r++)
printf("%s=%d ",RESOURCE_NAMES[r],zones[z].demand[r]);
printf("\\n");
}

printf("\\n Escalation Events:\\n");
for(int e=0;e<NUM_EVENTS;e++)
printf(" tick %2d -> Zone %d urgency +%d (%s)\\n",
EVENTS[e].tick,EVENTS[e].zone_id,EVENTS[e].delta,EVENTS[e].reason);
printf("\\n");
}

```

```
}
```

```
static void print_metrics_row(const char *label, Metrics m){  
    printf(" %-18s | %9.2f ticks | %8.3f | %10.1f%% | %10.1f%% | %8d | %6d\\n",  
        label, m.avg_response_time, m.fairness_index,  
        m.demand_satisfaction, m.supply_utilization,  
        m.starved_zones, m.ticks_to_finish);  
}
```

```
int main(void){  
    build_network();  
    precompute_all_paths();  
    print_scenario_summary();
```

```
    printf("=====  
    =====\\n");  
    printf(" STRATEGY COMPARISON\\n");
```

```
    printf("=====  
    =====\\n");
```

```
    printf(" %-18s | %14s | %8s | %11s | %11s | %8s | %6s\\n",  
        "Strategy", "Avg Response", "Fairness", "Demand Sat",  
        "Supply Util", "Starved", "Ticks");  
    printf(" -----+-----+-----+-----+-----+-----+-----  
    \\n");
```

```
    Metrics results[STRAT_COUNT];  
    for(int s=0; s<STRAT_COUNT; s++){  
        results[s]=run_strategy((Strategy)s);  
        print_metrics_row(STRATEGY_NAMES[s], results[s]);  
    }
```

```

printf("\n=====
=====\\n");

printf(" COMPOSITE EVALUATION\\n");

printf("=====
=====\\n");

double best=-1;
int best_idx=0;

for(int s=0;s<STRAT_COUNT;s++){
    double speed=100*(1-results[s].avg_response_time/MAX_TICKS);
    double fairness=results[s].fairness_index*100;
    double demand=results[s].demand_satisfaction;
    double composite=.35*speed+.35*fairness+.30*demand;

    printf("  %-18s composite = %6.2f (speed=%5.1f fairness=%5.1f demand=%5.1f)\\n",
           STRATEGY_NAMES[s],composite,speed,fairness,demand);

    if(composite>best){best=composite;best_idx=s;}
}

printf("\n-----\\n");
printf(" RECOMMENDATION: \"%s\"\\n",STRATEGY_NAMES[best_idx]);
printf(" Composite score: %.2f\\n",best);
printf("-----\\n");

return 0;
}

```